

Assignment 12

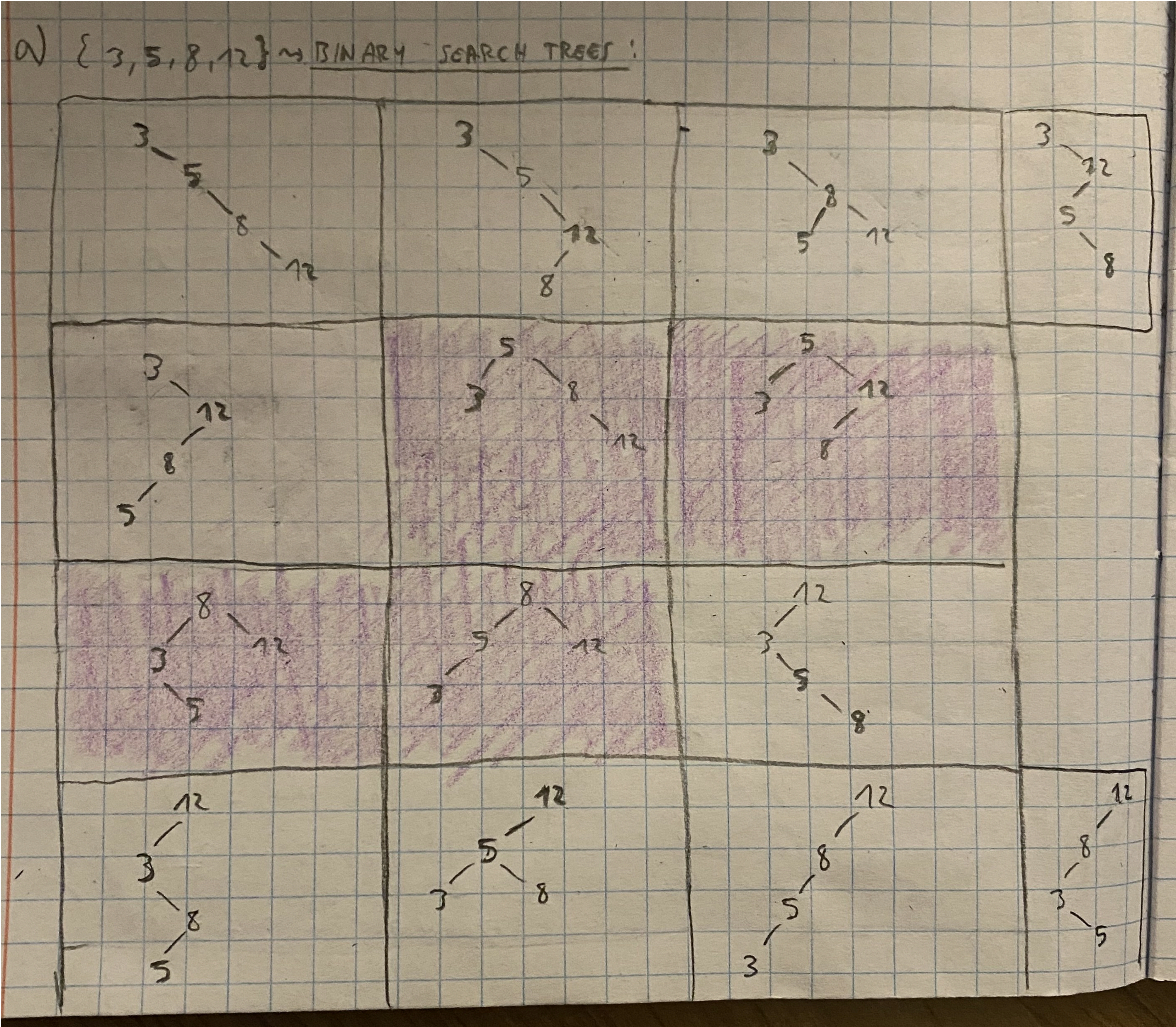
Author: Lan Stare

Date: 3. 12. 2025

Question 1

(a)

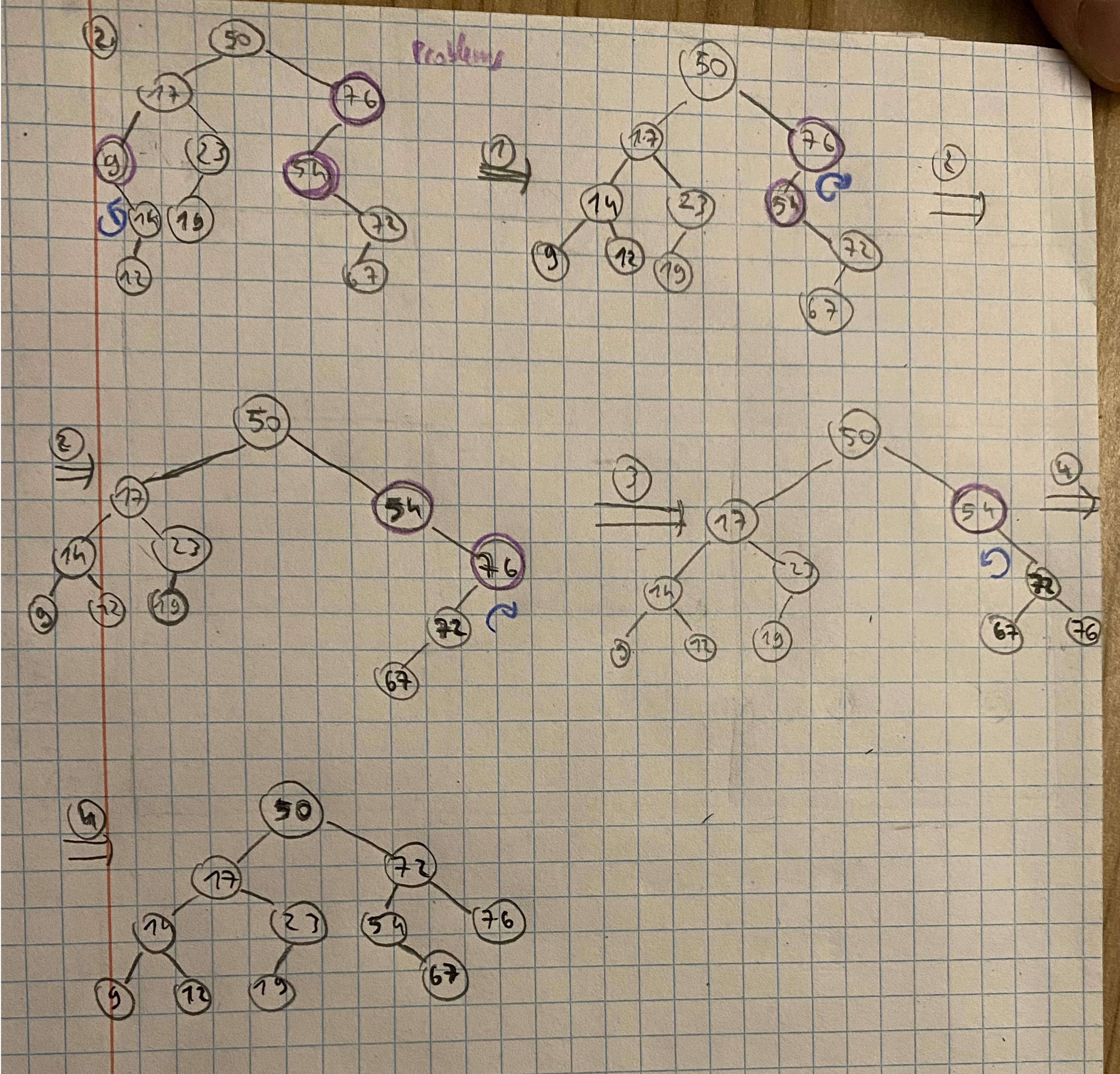
You can build 14 trees from 4 elements. All of them are drawn below:



(b)

There are 4 AVL trees you can build from 4 trees. All of them for our exercise are colored above :).

Question 2



Question 3

(a)

Let h denote the height of our tree. We have $m = 2^h - 1$ nodes. The maximum tree height is m , that is when we build the binary tree from a sorted array (either from 1 to $2^h - 1$ or from $2^h - 1$ to 1). Inserting an element to this kind of tree happens in $O(h) = O(m - 1) = O(2^h - 2) \in O(2^h)$ time. The reason why insertion has linear complexity is because we need to compare at most h amount of values to the value we're trying to insert. At each level of our tree we can either go left or right until we reach an empty spot.

(b)

In the lecture we showed that the minimum height of a tree with m nodes is $\log(m)$. So for our $m = 2^h - 1$, the height is $\log(2^h - 1)$. So one insertion happens in $O(\log(2^h - 1)) \in O(h)$ time.

Question 4

(a)

We can traverse the tree starting from the root like this: when we get to a node x , first compare k to $x.inodes$. If $x.inodes = (k - 1)$, we have found our k -th element. If $x.inodes > (k - 1)$, we take a left turn (we go to the left child of the node). Otherwise we go to the right. This way we take at most h turns (one at height 0 and none at height h). That's why this algorithm runs in $O(h)$.

The argument for correctness of this algorithm is that if we want to build a sorted array from a binary tree we start from the left most element (from the root we go left until we can't anymore; that's the smallest element). That element has no nodes in the left subtree. To get the second smallest element we first go to the right child of the previous node (if it has one) and then all the way to the left. If the right child does not exist, we go to the node's parent. This way we traverse the tree. With the algorithm above we can make finding k -th smallest element faster by comparing the number of nodes in the left child. That number says exactly how many smaller elements than the node are to the left of the node. So if we need an element that is bigger than that number, we can avoid traversing it.

(b)

The additional field only tells us how many elements are smaller than the current node but if we're trying to insert an element, we only care about the values of those nodes. More specifically, we only need to compare the current node to the element we're trying to insert. If the node is smaller than we take the right and if the node is bigger, we take the left. We have not used the additional property this binary tree has. That's why the algorithm for insertion is the normal insertion for binary trees which runs in $O(h)$.

Question 5

ALGORITHM AND EXPLANATION:

We can use the fact that we already have a sorted array. We can traverse the array once while filling the tree. Starting from the left most node, we assign the smallest value to that node. If it is possible, we then move to that node's right child and find the left most node and assign the second smallest value to that node. If the right child doesn't exist, we assign the second smallest value to the parent of the node with the smallest value. We do this for every node: either finding the minimum of the right child or assign the next value to the parent (we could derive a recursive equation for this).

COMPLEXITY = $O(n^2)$

We traverse the n -long array once and for every element we do at most $O(h)$ work (although usually a lot less). This is because if there is a right child we have to go to the left most element of the right child. In worst case this means that we go from the root to the bottom for every element. Because we don't specifically have a balanced trees, the biggest height of the tree can be $n - 1$, which means that our algorithm that runs in $O(nh)$ has upper limit at $O(n^2)$. If we had a balanced binary search tree, we could reduce this complexity to $O(n \log(n))$.